DYNAMO: Runtime Detection of Malicious Model Artifacts in the AI Supply Chain

***Abstract*—This document is a model and instructions for LaTeX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.**

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Malicious code poisoning in the AI model supply chain has emerged as a critical and rapidly growing threat. Unlike traditional software, model files are opaque binary artifacts routinely downloaded and executed with minimal inspection from platforms such as Hugging Face [1]. Recent attack reports reveal that compromised models have already caused large-scale theft of user credentials and illicit exploitation of LLM API tokens, affecting hundreds of thousands of end users and exposing CI/CD secrets of major service providers [2]–[4]. These incidents demonstrate that a single poisoned model artifact can propagate broadly through the reuse-driven AI ecosystem, amplifying the blast radius of each attack [5].

To defend against these threats, the majority of existing research has focused on static detection methods, which identify potential risks by scanning serialized files for known malicious signatures, decompiling pickle files, or detecting suspicious import behaviors [6]–[8]. These tools reason about code structure rather than executed behavior, so they cannot distinguish a benign model loading remote weights from a malicious payload establishing a reverse shell when both invoke the same networking library. Therefore there is a growing consensus that runtime detection is necessary to effectively mitigate these threats [9], [10]. However, existing dynamic detection methods rely on prior knowledge of attack patterns to instrument specific functions or filter system events, which makes them vulnerable to rapidly evolving adversarial techniques that embed payloads directly into computational graphs using only legitimate framework APIs [11]. This results in high false-positive and false-negative rates.

We identify the key limitation of existing defense strategies as their reliance on prior knowledge of attack patterns. Static detection tools scan serialized files for known malicious signatures, import dangerous function blacklists, or decompile pickle bytecodes to identify suspicious patterns. The problem is even more severe for dynamic methods [9]: instrumentation techniques hard-code which specific functions to hook based on expert heuristics, and system call monitors depend on static whitelists to filter benign events. Because adversarial techniques iterate rapidly any defense contingent on knowing what to look for is inherently vulnerable, suffering from both high false-positive and false-negative rates and complete defenselessness against zero-day exploits that lack established signatures.

To overcome this reliance on prior knowledge, we shift our strategy from hunting known threats to modeling benign behavioral characteristics. Our key observation is that benign AI models exhibit highly regular runtime templates, governed by the framework's internal logic and largely invariant across model architectures and tasks. Normal execution consistently passes through a small set of stable phases—deserialization, tensor materialization, and library loading—while malicious models inevitably deviate from these templates, producing anomalous patterns such as unexpected file access during deserialization or unauthorized network communication during inference.

Building on this observation, we propose DYNAMO, a defense that requires no prior knowledge of specific attack methodologies. DYNAMO transforms dynamic model security protection into a two-stage behavior analysis problem, bridging the semantic gap between low-level system evidence and model intent. In the first stage, an unsupervised model learns the distribution of benign operation patterns from system call traces and efficiently filters out normal activity, requiring no attack signatures or whitelists. Only statistically abnormal operations are escalated to the second stage, where they are associated with high-dimensional Python semantic context collected via lightweight, non-invasive instrumentation. This cross-layer association combines non-bypassable system call evidence with application-layer semantics. Finally, the LLM is used to trace the root cause of underlying operations and judge whether such behaviors conform to the regular execution logic of the model or belong to malicious attacks. Through this two-stage pipeline, DYNAMO achieves both high detection accuracy and low false-positive rates while maintaining practical efficiency.

However, realizing DYNAMO faces three non-trivial technical challenges. First, how to define a meaningful unit of system call behavior that can be learned by a machine learning model. A single system call is excessively fine-grained and massive in volume, while malicious behaviors are generally manifested via consecutive correlated call sequences. DYNAMO addresses this by aggregating system calls into lifecycle-bounded operations, converting raw traces into semantically meaningful operation vectors. Second, how to achieve comprehensive semantic instrumentation across heterogeneous, rapidly evolving AI frameworks without requiring source-code access or

imposing prohibitive engineering overhead. DYNAMO leverages an LLM to dynamically generate framework-adaptive monkey-patching scripts, which automatically discover and wrap all public callable functions at runtime. Third, how to reliably align asynchronous Python-level semantic events with kernel-level system call traces under high concurrency, where naive timestamp matching produces spurious associations. DYNAMO addresses this through a fuzzy spatio-temporal alignment strategy that combines temporal windowing with resource-related spatial hints to robustly associate each abnormal operation with its true application-layer context.

Experiments on 89 real-world and 80 advanced synthetic malicious models demonstrate that DYNAMO achieves 100% precision, 98.70% F1-score, and 0% false positives on benign models using sensitive framework APIs, while introducing only 5.69% average runtime overhead. In summary, our contributions are as follows:

- We propose the first hybrid dynamic detection system for AI model supply chain attacks that combines non-bypassable system-call evidence with Python semantic context, bridging the semantic gap between low-level behaviors and model intent.
- We develop a robust cross-layer association mechanism that aligns abnormal system-level operations with high-level semantic events, enabling LLM-based reasoning to provide accurate and explainable security alerts.
- Extensive experiments on real-world and synthetic malicious models demonstrate that our system achieves high detection accuracy and low false positive rates, outperforming existing static defenses with good practical deployment value.

II. Background

A. AI Model Supply Chain Attack

AI model supply chain. The growth of open-source model hubs has transformed AI model development, enabling the community to freely download, fine-tune, and integrate pre-trained models into downstream applications [12]. As of 2026, Hugging Face hosts over 2.8 million models spanning 1,000+ languages and dozens of tasks [13], [14]. Foundational models like BERT and GPT have become indispensable, each recording over ten million downloads, demonstrating the critical role of the model supply chain in advancing research and industry [15].

AI model supply chain attack. This sharing paradigm, while facilitating collaboration, introduces severe security vulnerabilities to the supply chain. Recent reports have uncovered structural vulnerabilities on platforms such as Hugging Face [16], and real-world attacks targeting this supply chain have surged [17]. As illustrated in Figure 1, the typical attack workflow involves uploading compromised models to public hubs that execute hidden payloads when downloaded and deployed. These attacks have already resulted in large-scale theft of user credentials and illicit exploitation of LLM API tokens, causing significant financial and operational damage [2].

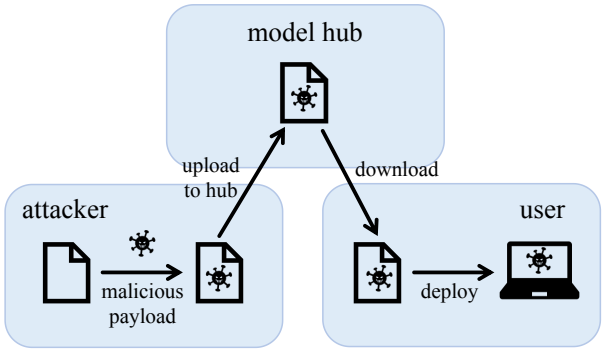

Fig. 1: The workflow of an AI model supply chain attack.

AI model supply chains are vulnerable to such attacks because malicious code is embedded stealthily within opaque binary artifacts []. Attackers exploit this by embedding payloads directly into model structures using obfuscation techniques [18]. For instance, malicious code can be disguised as normal model components or hidden within unreadable binary formats to evade static security scanners [19].

Recent studies provide comprehensive analyses of malware poisoning in pre-trained model hubs [20], with particular attention to insecure serialization formats [21] that embed arbitrary execution commands triggered automatically upon model loading. Since payload execution is integrated into the normal deserialization process, it bypasses conventional detection methods [22].

B. Existing Attack Vectors

Current attack methodologies targeting the AI model supply chain fall into two categories: insecure serialization attacks that embed executable payloads in model file formats, and framework-native attacks that weaponize legitimate framework APIs under the guise of normal tensor computations.

Insecure serialization attacks arise from a fundamental trade-off between architectural flexibility and system security. To preserve complex model architectures alongside learned weights, frameworks rely on serialization mechanisms supporting arbitrary object instantiation [23], blurring the boundary between passive data and executable code [24]. Consequently, loading a compromised artifact triggers automatic payload execution before any ML computation. Recent studies confirm that insecure serialization remains widespread on public hubs, and malicious artifacts can propagate through ordinary sharing workflows [25], [26].

The vulnerability level depends on the storage paradigm. Formats preserving both architecture and weights pose the highest risk. Python's pickle and its derivatives, widely adopted by PyTorch and Scikit-learn, support arbitrary object instantiation and are highly vulnerable [27], [28]. TensorFlow's SavedModel and Keras's HDF5 [29] encapsulate execution graphs or custom layer definitions, presenting attack surfaces when loading untrusted artifacts. In contrast, formats restricted to raw numerical weights are inherently secure: Hugging Face's Safetensors [30] strictly parses flat tensor data,

and ONNX [31] relies on a rigid schema. Table I summarizes the security posture of widely used serialization formats.

TABLE I: model formats and their vulnerability to code injection attacks.

Model Format	Primary Frameworks	Vulnerability Level
`pickle`, `marshal`	PyTorch, Scikit-learn	Highly Vulnerable
`joblib`, `dill`	Scikit-learn, Ray, PySpark	Highly Vulnerable
`SavedModel`	TensorFlow	Potentially Vulnerable
`HDF5`	Keras	Potentially Vulnerable
`Checkpoint`	TensorFlow	Secure
`Safetensors`	Hugging Face	Secure
`ONNX`	ONNX	Secure
`JSON`, `MsgPack`	Various	Secure

Framework-native attacks weaponize legitimate framework APIs to execute malicious operations without introducing obvious malicious syntax. Malicious logic is compiled directly into the computational graph rather than embedded as standalone scripts. Frameworks like TensorFlow offer lower-level APIs for distributed training and debugging that inherently possess file system access and network communication capabilities [32]. Attackers hijack these operations—for example, using debugging APIs to exfiltrate local data. The TensorAbuse framework [11] systematically demonstrates this paradigm by embedding dangerous operators directly into the inference pipeline, enabling file exfiltration, IP exposure, code injection, and remote shell acquisition under the guise of normal tensor operations.

This technique is exceptionally stealthy. Traditional static analysis scans for known malicious signatures or external library imports, but framework-native attacks generate no external dependencies and easily bypass static blacklists. Malicious operations remain dormant during loading and trigger only during inference as tensors flow through the graph. This execution dynamic creates a severe semantic gap: without application-layer context, security monitors cannot distinguish legitimate data loading from malicious file exfiltration.

C. Existing Defenses and Their Limitations

While extensive research addresses inference-time threats such as adversarial examples and data poisoning, efforts to secure the AI supply chain against malware injection remain limited. Current defenses primarily rely on static analysis and signature-based scanning.

Fickling [7] decompiles pickle bytecodes into readable Python code to identify embedded malicious payloads. Hugging Face employs Picklescanning [8], which combines ClamAV [33] with import table inspection within pickle files. A static detection methodology for suspicious API calls in computational graphs was also proposed alongside TensorAbuse.

The foremost limitation of these tools is their reliance on static pattern matching—detecting predefined libraries or suspicious function calls—without analyzing dynamic execution or establishing semantic context. A benign component fetching remote weights and a malicious payload establishing a reverse shell might both use the same network library. This lack of runtime semantic analysis inevitably leads to high rates of both false positives and false negatives.

Furthermore, these tools are inherently reactive, lacking a comprehensive understanding of evolving attack vectors. They are engineered to address specific, previously observed vulnerabilities rather than adopting a holistic approach. This severely restricts their ability to defend against zero-day exploits or novel attacks that do not match known signatures.

III. Threat Model

In this paper, we focus on malicious code poisoning attacks within the AI model supply chain, in which malicious actors exploit the collaborative nature of pre-trained model hubs to distribute compromised artifacts and compromise victim systems.

Attackers. The attackers are malicious AI model providers or actors who have compromised legitimate developer accounts [34]. They have the capability to create and upload malicious datasets and binary model files to public model hubs without providing the original training source code. To embed their payloads, attackers can exploit insecure serialization formats [28] or abuse legitimate, hidden capabilities of framework APIs [11] to bypass static security scanners. They may also employ techniques like exploiting leaked authentication tokens or hijack trusted identities, thereby deceiving the community [35]. Their primary goal is to execute unauthorized actions, such as establishing reverse shells, exfiltrating sensitive data such as user credentials, or deploying ransomware, without attracting attention.

Victim Users. The victims are AI developers and researchers who seek to download and reuse pre-trained models or datasets for their applications []. They generally trust content hosted on well-known model hubs and popular contributors, often prioritizing convenience and efficiency over rigorous security inspections. Victims typically load and execute these models using standard libraries and standard user-level permissions, rather than sandboxed environments. During the standard model loading or inference processes, they unknowingly trigger the embedded malicious code or behaviors [5].

Model Hubs. Model hubs represented by Hugging Face and TensorFlow Hub serve as core platforms for the centralized storage, retrieval, and distribution of AI resources. While they implement certain security measures, such as basic malware scanning, these defenses often lag behind rapidly evolving threats and fail to perform deep semantic-level analysis. The platforms treat models as opaque binary files and typically guide users to download and run them directly without sandboxing. Consequently, they struggle to detect sophisticated attacks that disguise malicious code as normal model components or abuse legitimate framework operations.

IV. Approach

In this section, we present our approach for detecting malicious behaviors embedded in model artifacts. The pipeline operates in three stages: (i) *Data Collection*, which gathers two synchronized evidence streams—low-level system-call traces

TABLE II: Security-relevant system-call categories used by our filter.

Filter param	Category	Purpose
file	File operations (`read`/`write`)	Detect sensitive data leakage or configuration file tampering
network	Network operations (`socket`)	Detect data exfiltration or remote command execution
cred	Credential operations (`setuid`)	Detect privilege-escalation behavior
desc	Descriptor operations (`dup`)	Assist file-related anomaly detection
signal	Signal handling (`kill`)	Capture malicious process termination attempts
clock	Time-related operations	Assist temporal anomaly analysis (e.g., abnormal delays)

and high-level Python semantic logs; (ii) *Coarse-grained Filtering*, which clusters raw system calls into operation vectors, screens for anomalies via an unsupervised autoencoder, and retrieves aligned semantic context through fuzzy spatio-temporal matching; and (iii) *Fine-grained Verification*, where an LLM jointly reasons about the system-call evidence and Python semantic context to produce an intent score, a threat category, and an auditable evidence chain.

A. Data Collection

The data collection stage simultaneously captures two synchronized streams of evidence during controlled model execution: a kernel-visible system-call stream and an application-layer semantic log stream. Together, these streams provide the complementary low-level and high-level signals needed for downstream analysis.

1) System-call collection.: We base our detection on system-call traces because they operate below the user-space level and cannot be bypassed by any Python-level obfuscation [36]: no matter how a model artifact hides its malicious intent through code mangling or dynamic loading, its actual interactions with the operating system must traverse the kernel and therefore leave a non-bypassable forensic record.

We employ a selective tracing strategy that focuses exclusively on security-relevant system-call families, discarding high-frequency but semantically uninformative calls to keep the trace volume manageable. During controlled model execution, we use `strace` to continuously trace system calls of the target process. To reduce noise and overhead, we immediately discard calls such as scheduler hints and memory-mapping operations that carry no security signal [37]. The selection concentrates on system-call categories that can indicate malicious intent. Table II summarizes the security-relevant syscall families we monitor and their roles in our detection pipeline. For example, file operations can reveal attempts to access sensitive data or modify configurations, while network operations may signal data exfiltration or remote command execution.

2) Instrumentation log collection.: To bridge the semantic gap between model-level intent and kernel-level behavior, we collect high-dimensional Python semantic context via non-invasive instrumentation. The key challenge is that heterogeneous ML frameworks expose frequently changing APIs, making manual per-framework instrumentation labor-intensive and coverage-incomplete. We address this through an LLM-assisted approach with two distinct phases.

Algorithm 1: Automated Dynamic Instrumentation Logic

Input: Target library module $\mathcal{M}$ (e.g., `torch`, `tensorflow`)
Output: Instrumented module capable of context logging

Procedure `ApplyDynamicPatches`($\mathcal{M}$)**:**
  foreach *attribute name* $attr\_name$ **in** dir($\mathcal{M}$) **do**
    if $attr\_name$ *starts with "_"* **then**
      continue
    $target\_func \leftarrow$ getattr($\mathcal{M}, attr\_name$);
    if not *is_callable*($target\_func$) **then**
      continue
    $wrapper \leftarrow$ `Wrapper`($target\_func, attr\_name$);
    try;
      setattr($\mathcal{M}, attr\_name, wrapper$);
    catch *ReadOnlyAttributeError* ;
      continue ;
Procedure `Wrapper`($func, name$)**:**
  return a closure that captures the full `traceback`, logs to JSON, executes $func$, and delegates return value;

Pre-execution: API Discovery and Script Generation. Before model execution, we leverage an LLM to synthesize dynamic instrumentation scripts. A rigorously structured prompt constrains the LLM to generate code that inspects the target module's namespace at startup, systematically identifies all public callable attributes, and constructs wrapper closures around each. The prompt specifies the security objective, mandates capturing execution context (timestamps, operation types, argument values, return types, and full call stacks via `traceback`), and requires all intercepted events to be serialized into standardized JSON. This automated discovery eliminates statically hardcoded API lists and adapts to framework version changes by simply regenerating wrappers for the new API surface, without modifying the core detection pipeline.

Algorithm 1 formalizes this discovery and wrapping logic. The `ApplyDynamicPatches` procedure isolates public callables while skipping private attributes and non-callable objects, then generates wrapper closures for each target API.

Runtime: Wrapper Injection and Context Logging. At process startup, the generated script employs monkey patching (`setattr`) to replace original functions with wrapper closures. Thereafter, any invocation of a wrapped API triggers the injected closure, which captures the execution context and serializes it as a structured semantic event:

$$\ell_i = \langle \textit{FuncName},\ \textit{ArgsSummary} \rangle. \tag{1}$$

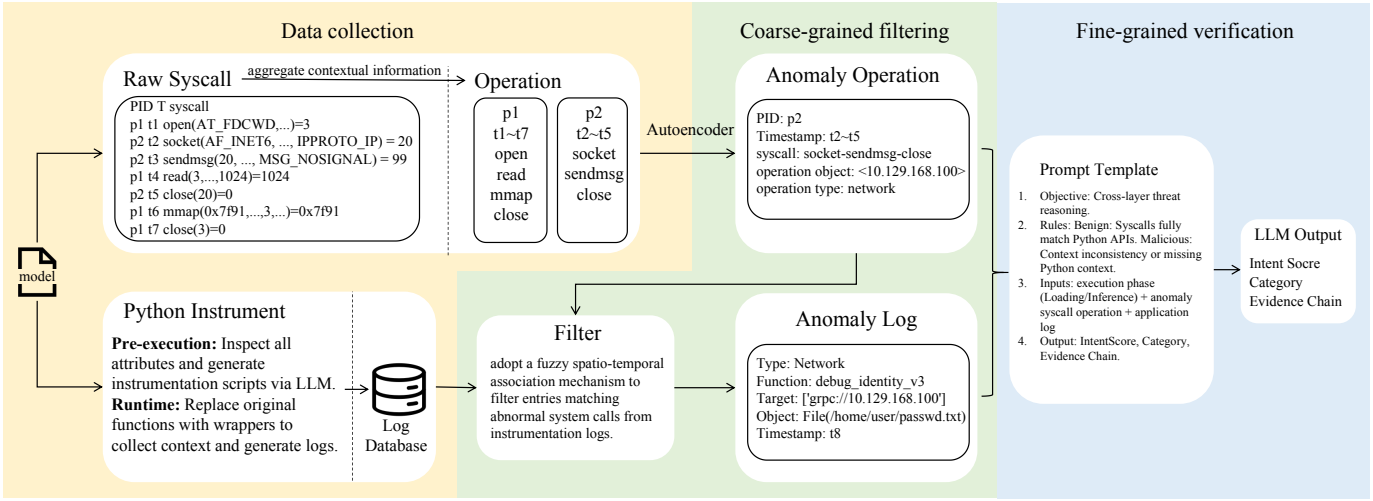

Fig. 2: Overview of the hybrid defense system.

Here *FuncName* denotes the invoked framework or standard-library API, while *ArgsSummary* aggregates only lightweight information—tensor shapes and dtypes, configuration flags, truncated or hashed file paths, and bounded-length string summaries—controlling logging overhead while preserving rich contextual signals.

To ensure runtime safety, wrappers follow a conservative policy: they only wrap callable functions (avoiding class definitions, built-in types, and C/C++ bindings), perform best-effort argument summarization without altering return values or side effects, and employ defensive serialization (e.g., recording shapes instead of raw tensor contents). The patching process is additionally wrapped in `try-except` blocks to survive read-only properties, immutable C-extension bindings, and dynamically generated descriptors that would otherwise crash the process. Any patch failure degrades gracefully without disrupting the target process.

B. Coarse-grained Filtering

After collecting raw evidence streams, the coarse-grained filtering stage transforms system-call traces into compact operation vectors, screens for anomalies, and retrieves the corresponding semantic context. This stage eliminates clearly benign activity before the expensive LLM reasoning stage.

1) System-call Clustering and Embedding: The collected system-call traces are extremely fine-grained and voluminous. A single model loading process may produce thousands of system calls, and meaningful malicious behavior invariably manifests as a *sequence* of related calls rather than as isolated events. To address this, we aggregate correlated system calls into compact, high-level operation vectors that reduce trace noise, compress data volume, and provide standardized analysis units with complete semantics for downstream anomaly detection.

To accurately model the execution semantics of a program, it is crucial to recognize that system calls inherently possess varying degrees of contextual dependency. Applying a uniform extraction strategy to all system calls would be ineffective: some operations are semantically self-contained and immediately indicative of high-risk behavior, whereas others are granular, stateful operations that remain ambiguous unless analyzed as a continuous sequence. Motivated by this fundamental difference, we propose a hybrid embedding scheme that categorizes the traced system calls into two distinct streams for targeted handling: (1) *Directly malicious calls* and (2) *Stateful calls*. By processing these two streams individually, we ultimately synthesize them into a unified feature space for downstream analysis.

Direct call embedding. The first category includes standalone, high-risk operations (e.g., `execve`, `chmod`) that do not require lifecycle context. We directly encode each of these calls into a fixed-dimension operation vector $\mathbf{v}_{dir} \in \mathbb{R}^d$. Specifically, we utilize a pre-trained BERT-based model to extract the rich semantic meaning from the call's underlying logic and literal arguments. This high-dimensional representation is subsequently projected down to the target dimension d, ensuring that semantically critical, self-contained threats are instantly captured without the need for sequence aggregation.

Stateful aggregation. Let $S = \{(s_k, \tau_k)\}_{k=1}^{N}$ denote the traced system-call stream, where s_k is the k-th syscall and τ_k its timestamp. For stateful operations, we maintain per-resource state keyed by $r = \langle PID, FD \rangle$, where *FD* denotes a file or socket descriptor. For each key r, we collect the indices of calls that operate on this resource within one lifecycle segment,

$$I_r = \{\, k \mid (s_k, \tau_k) \in S,\ s_k \text{ uses resource } r,\ \tau^r_{start} \le \tau_k \le \tau^r_{end} \,\}, \tag{2}$$

where τ^r_{start} and τ^r_{end} are the timestamps of the resource-acquisition and resource-release calls (e.g., `open` / `close` or `socket` / `shutdown`). The lifecycle segment for resource r is then the ordered subsequence $S_r = \{(s_k, \tau_k)\}_{k \in I_r}$.

We aggregate this segment into a fixed-length operation vector by applying a feature-extraction function ϕ over all calls

in the segment:

$$\mathbf{v}_{sys}(r) = \phi(S_r) \in \mathbb{R}^d, \tag{3}$$

where ϕ operates by extracting the common resource identifier $r = \langle PID, FD \rangle$ shared across the segment S_r, and concatenating it with the sequence of distinct, call-specific parameters (e.g., flags, buffer sizes, or return values) derived from each individual system call $s_k \in S_r$. This concatenated feature sequence is subsequently passed through an embedding layer to project it into the continuous $\mathbb{R}^d$ space. Intuitively, each $\mathbf{v}_{sys}(r)$ captures a dense, contextualized representation of the exact data flow and state transitions associated with a specific file or connection during its lifetime, effectively smoothing out scheduling artifacts and interleaving from other threads.

The final feature matrix combines $\mathbf{v}_{dir}$ and $\mathbf{v}_{sys}$, enabling the system to capture both direct and aggregated syscall behaviors. This hybrid representation ensures that directly malicious actions are immediately flagged, while stateful operations are analyzed in context to detect more subtle threats.

2) Autoencoder-based Anomaly Detection: We adopt unsupervised anomaly detection because benign model loading and inference produce a stable, learnable distribution of system-call operations, whereas malicious behaviors are inherently rare and diverse, making supervised classification impractical without labeled attack samples that cover the full threat space. Among unsupervised methods, we choose an Autoencoder for its conceptual simplicity and proven effectiveness in reconstruction-based anomaly detection: it learns to faithfully reconstruct benign operation vectors during training and naturally flags any vector it cannot faithfully reconstruct as anomalous.

We train an Autoencoder on operation vectors collected from clean model loading and inference runs. Given an operation vector $\mathbf{v}$, the Autoencoder computes a reconstruction error

$$E = \|\mathbf{v} - Dec(Enc(\mathbf{v}))\|_2^2, \tag{4}$$

where *Enc* and *Dec* denote the encoder and decoder networks, respectively. Operations with reconstruction error E below a threshold θ are treated as consistent with normal model behavior and are discarded. If $E > \theta$, we mark the corresponding operation as abnormal, retain its full underlying system-call segment S' and metadata $\langle PID, TID, t_{start}, t_{end} \rangle$, and promote it to the subsequent spatio-temporal matching stage.

By performing this unsupervised prefiltering at the operation level, the system keeps the expensive cross-layer association and LLM reasoning off the critical path for the vast majority of benign activity, while still surfacing rare, suspicious low-level behaviors that deviate from the learned distribution of normal AI model execution.

3) Spatio-temporal Semantic Matching: After the autoencoder identifies abnormal operations, we must determine whether each anomaly can be explained by benign Python-level behavior. This requires retrieving the semantic context from the instrumentation log stream that corresponds to each abnormal system-call operation.

System calls and Python semantics are inherently asynchronous due to operating system scheduling and concurrent I/O. As a result, naive one-to-one timestamp matching between system-call logs and semantic logs is unreliable in high-concurrency workloads. We therefore adopt a fuzzy spatio-temporal association strategy.

For each abnormal operation, the autoencoder screening stage provides the underlying system-call segment S' and its metadata $\langle PID, t_{start}, t_{end} \rangle$. Given this metadata and the semantic log stream $L = \{(\ell_i, t_i)\}$, we perform filtering following the spatial-temporal matching rules below.

Temporal windowing. We apply a slack temporal window around the abnormal operation. For each candidate abnormal operation, we define its active time span $[t_{start}, t_{end}]$ based on the first and last system call in S', and search semantic events whose timestamps t_i fall into an expanded window $[t_{start} - \Delta t, t_{end} + \Delta t]$. The slack parameter Δt tolerates minor scheduling jitter while still localizing semantic context to the period when the resource was actually in use.

Spatial hints. Temporal proximity alone is insufficient under heavy concurrency. We therefore further filter candidates using resource-related hints derived from both layers, such as whether the operation is file- or network-related, file paths or prefixes, remote destinations, and other descriptor metadata when available. Combining temporal and spatial constraints yields a fuzzy but robust alignment that significantly reduces spurious associations when many model components execute in parallel.

Cross-layer behavior profile. After spatio-temporal filtering, we bundle the abnormal operation vector, its raw system-call summary, and the aligned semantic events into a structured cross-layer behavior profile. This profile serves as the input to the LLM-based intent reasoning stage: it contains (i) what the OS observed at the system-call level, (ii) which high-level Python APIs and call sites were active around that time and on the same resource, and (iii) basic execution-phase information. Together, these elements provide sufficient context to decide whether the abnormal low-level behavior is consistent with benign model execution or indicative of malicious intent.

C. Fine-grained Verification

Operations that survive coarse-grained filtering are escalated to the fine-grained verification stage, which jointly reasons about the low-level system-call evidence and high-level semantic context to produce a final intent judgment.

Determining whether abnormal low-level operations reflect benign model behavior or malicious intent is inherently challenging: the same system-call pattern can arise from legitimate model loading or from an attacker's attempt to exfiltrate data, and without semantic context the two are indistinguishable at the kernel level. To resolve this ambiguity, we feed each abnormal operation together with its aligned semantic contexts into an LLM, which judges whether the behavior is consistent with benign model execution.

Prompt construction. We translate the aligned evidence into a structured natural-language prompt that includes: (i) a concise

summary of the abnormal operation including operation type, key statistics, and risk-relevant parameters, (ii) the aligned semantic call contexts including library/API, call stack summary, and argument summaries, and (iii) the execution phase categorized into the model loading stage and inference stage.

Our maliciousness judgment follows a principled cross-layer reasoning rule: a behavior is classified as benign only when its low-level system-call activity can be plausibly explained by the aligned Python-level semantic context. Concretely, if the LLM identifies that the observed file reads, network connections, or process creations are consistent with the framework APIs and execution phase reported in the semantic log, the operation is deemed benign and discarded. Conversely, when inconsistencies exist between the Python-layer semantic logs and the corresponding system-call activities, such as a network connection to an external host with no matching high-level API invocation, or when no matching Python semantic context can be retrieved at all, the behavior is marked as malicious and escalated.

Decision and explanation. The LLM outputs *IntentScore*, a *ThreatCategory* label, and a rationale. Importantly, we persist the subset of semantic events and system-call summaries used in the reasoning as the *evidence chain Context* for auditing. If high-risk system-call patterns appear without any plausible semantic explanation, the system raises a high-severity alert and can block execution in the controlled environment.

D. Model Vetting Pipeline

DYNAMO is designed to operate in a pre-deployment model vetting pipeline rather than in production inference serving. Before a model is released or integrated into any production system, it should undergo a comprehensive security vetting phase in a controlled sandbox environment with DYNAMO monitoring its behavior to detect embedded malicious logic.

To maximize detection coverage, we recommend exhaustive fuzzing-based testing of all real-world business workflows that the model will encounter post-deployment. Concretely, the model should be exercised through every anticipated usage scenario: loading with all expected input shapes and data types, invoking inference across the full range of supported batch sizes, exercising all serialization, checkpointing, and fine-tuning paths, and simulating edge-case inputs that stress runtime boundaries. This fuzzing strategy ensures that dormant malicious payloads—which may be conditionally triggered only under specific runtime conditions, input patterns, or execution phases—are activated and exposed during testing, providing comprehensive coverage beyond simple model loading.

V. Evaluation

We evaluate the effectiveness of DYNAMO in detecting malicious behavior embedded in model artifacts. Our study focuses on three aspects: (i) detection performance compared to state-of-the-art defenses, including robustness to camouflage attacks such as TensorAbuse; (ii) the impact of cross-layer modeling and its components on accuracy and explainability; and (iii) runtime overhead and deployability in realistic analysis environments. We organize the evaluation around the following research questions:

- **RQ1:** What is the performance of DYNAMO in detecting malicious models, and does it show significant advantages over the baselines?
- **RQ2:** What is the runtime overhead of DYNAMO, and is it practical to deploy in real-world model workflows?
- **RQ3:** How does each part of DYNAMO contribute to the detection performance?

A. Experiment Setup

Datasets. We evaluate DYNAMO on two datasets spanning known deserialization exploits and advanced framework-native attacks:

- *MalHug Dataset.* 89 models identified as anomalous by MalHug from public repositories. Manual source code auditing and sandbox verification label 78 as malicious—mainly abusing Pickle deserialization to implant reverse shells—and 11 as benign proof-of-concept samples that verify exploitability without destructive operations. While the malicious models represent real-world threats, their methodologies rely on well-known deserialization flaws and are straightforward to detect with existing static scanners.
- *Advanced TensorAbuse Synthetic Dataset.* We construct a second dataset using TensorAbuse to evaluate against stealthy, framework-native attacks. The **malicious subset** contains 80 models: for each of 10 diverse foundation models, we embed 8 attack variants spanning four categories—File Exfiltration, IP Exposure, Malicious Code Insertion, and Remote Shell Acquisition—using TensorAbuse primitives. The **Benign Subset** contains 40 models that invoke the same high-risk native TensorFlow APIs for legitimate purposes (loading dataset configurations, outputting debug tags, standard tensor persistence), constructed to precisely quantify the false-positive rate.

For all datasets, the model artifact is the unit under test. Labels are assigned by manual payload inspection and sandboxed verification of malicious operations.

Baselines. We compare against four baseline detection methods: three widely adopted security scanners integrated into the Hugging Face platform—JFrog [38], Protect AI [6], and ClamAV [33]—and the specialized detection tool from TensorAbuse [11].

For JFrog, Protect AI, and ClamAV, we uploaded our model artifacts to the Hugging Face platform, triggering its live security scanning pipelines to capture authentic detection efficacy in a real-world supply chain setting. For TensorAbuse, which statically identifies suspicious API calls in computational graphs, we use the authors' open-source implementation. Since these baselines rely on static analysis and signature matching without dynamic execution, we evaluate them directly against raw model artifacts under authentic, platform-native conditions.

TABLE III: Comprehensive detection performance of DYNAMO and baselines across both datasets. MalHug Dataset metrics are computed on 78 malicious and 11 benign models; Advanced TensorAbuse Synthetic Dataset metrics are computed on 80 advanced malicious models with false positives evaluated on 40 benign TensorFlow-API models.

Method	MalHug Dataset			Advanced TensorAbuse Synthetic Dataset		
	Precision	Recall	F1	Precision	Recall	F1
JFrog	0.8953	0.9872	0.9390	0.8889	0.3750	0.5275
Protect AI	0.8966	**1.0000**	0.9455	0.7273	0.5000	0.5928
ClamAV	0.8904	0.8333	0.8609	–	0.0000	–
TensorAbuse	–	–	–	0.6667	**1.0000**	0.8000
DYNAMO	**1.0000**	0.9744	**0.9870**	**1.0000**	**1.0000**	**1.0000**

Implementation Details. DYNAMO is implemented in Python. The kernel-level module uses `strace` to collect system calls from the target process and its children, aggregates raw traces into per-resource operation vectors $\mathbf{v}_{dir}$ and $\mathbf{v}_{sys}$ (Section IV), and trains an Autoencoder on vectors from trusted benign models with the reconstruction error threshold θ chosen on a validation set.

For Python instrumentation, we apply non-invasive monkey patching to framework and standard-library APIs for security-relevant operations (file I/O, network, subprocess). Each invocation records a structured event $\ell_i = \langle PID, TID, FuncName, ArgsSummary \rangle$ with bounded argument summaries. Cross-layer association aligns abnormal operations with nearby semantic events via the spatio-temporal strategy in Section IV, producing behavior profiles fed to an LLM for intent reasoning. We use a production-grade LLM with temperature zero for deterministic decisions.

All experiments run on a workstation with dual Intel Xeon Silver 4210R CPUs, 128 GB RAM, and two NVIDIA GeForce RTX 3090 GPUs.

B. RQ1: Detection Performance

To answer this research question, we evaluate the detection performance of DYNAMO and the baseline methods across the MalHug Dataset and the Advanced TensorAbuse Synthetic Dataset. This comprehensive evaluation demonstrates DYNAMO's effectiveness in identifying various malicious activities, including known deserialization exploits and sophisticated attacks implemented by leveraging native framework capabilities.

Table III summarizes the detection performance across both datasets. On the MalHug Dataset, traditional static scanners achieve moderate to high F1 scores by matching known deserialization signatures. DYNAMO achieves a marginal performance improvement over these tools via accurate intent verification. However, on the Advanced TensorAbuse Synthetic Dataset, the performance gap expands dramatically. Static tools suffer a sharp drop in detection performance. While TensorAbuse achieves a perfect recall, its precision is only 0.6667, as it fails to distinguish legitimate and malicious calls to TensorFlow APIs. Only DYNAMO maintains consistently high performance on both datasets, achieving an F1 of 0.9870 on the MalHug Dataset and a perfect 1.0000 on the Advanced TensorAbuse Synthetic Dataset. We next analyze each dataset in detail.

TABLE IV: Number of detected malicious models by DYNAMO and baselines on the Advanced TensorAbuse Synthetic Dataset, categorized by attack type (20 models per type, 80 models total).

Method	Attack Type (Total models)				Total Detected
	File Exfil.	IP Exposure	Code Insertion	Remote Shell	
JFrog	20 / 20	0 / 20	0 / 20	10 / 20	30 / 80
Protect AI	20 / 20	0 / 20	0 / 20	20 / 20	40 / 80
ClamAV	0 / 20	0 / 20	0 / 20	0 / 20	0 / 80
TensorAbuse	**20** / 20	**20** / 20	**20** / 20	**20** / 20	**80** / 80
DYNAMO	**20** / 20	**20** / 20	**20** / 20	**20** / 20	**80** / 80

Performance on the MalHug Dataset. Traditional static detection tools such as JFrog, Protect AI, and ClamAV achieve relatively high recall rates by relying on established vulnerability signatures. However, these baseline methods exhibit a critical limitation: they fail to distinguish between actual weaponized attacks and the 11 benign proof-of-concept models. By simply classifying any model containing a vulnerable signature as malicious, these traditional methods generate substantial false positives.

In contrast, DYNAMO successfully detects 76 out of the 78 genuinely malicious models while accurately categorizing all 11 proof-of-concept models as non-malicious. Because DYNAMO relies on a two-stage screening process that evaluates the actual intent of dynamic behavior rather than merely matching static signatures, it demonstrates robust resilience against false positives. Notably, the TensorAbuse detector fails to identify any of these models. Since it is built upon a static graph-based solution, it cannot detect deserialization exploits triggered before computational graph initialization or outside the scope of the computational graph.

Performance on the Advanced TensorAbuse Synthetic Dataset. Table IV provides a detailed breakdown of the detection results. Both DYNAMO and the TensorAbuse detection tool successfully detect 100% of the malicious models. The TensorAbuse tool achieves perfect detection here because these specific attacks were generated using its own primitive methodologies, making them highly visible to its static API-checking mechanism. However, as demonstrated in Table III, its static approach fails entirely against attacks outside its predefined scope.

Conversely, DYNAMO achieves this 100% detection rate without relying on static signatures or predefined API lists. By statistically profiling the benign baseline behaviors of models and dynamically intercepting the anomalous system calls generated by malicious payloads, DYNAMO accurately identifies malicious intent regardless of the specific attack vector. The performance of JFrog, Protect AI, and ClamAV varies significantly across attack categories, highlighting the limitations of traditional scanning tools against sophisticated, framework-native manipulations.

To further verify DYNAMO's capability against false positives, we also evaluate all methods on benign models.

The precision results on the Advanced TensorAbuse Synthetic Dataset in Table III clearly reveal the drawbacks of static detection mechanisms. TensorAbuse misclassifies all 40 benign models as malicious, resulting in a 100% false positive rate. It relies solely on static scanning against a predefined blacklist of high-risk APIs and lacks the ability to understand API semantics and invocation context. Similarly, JFrog and Protect AI also produce numerous misclassifications. This indicates that signature-based detection methods fail when legitimate operations overlap with known attack patterns.

In contrast, DYNAMO successfully identifies all 40 models as benign. By dynamically observing runtime execution, DYNAMO's first stage recognizes that the system calls generated by these APIs align with the established statistical baseline of normal framework operations. When a suspicious operation triggers cross-layer analysis, the LLM-based reasoning engine successfully interprets high-level Python semantics to validate benign intent. This demonstrates that DYNAMO bridges the semantic gap, providing accurate defense without impeding legitimate workflows.

Overall, experimental results on both datasets prove that DYNAMO is a robust defense solution with strong generalization capability. It can accurately detect malicious AI models exploiting common deserialization vulnerabilities or sophisticated and stealthy computational graph tampering. Furthermore, equipped with cross-layer semantic reasoning, DYNAMO effectively differentiates legitimate usage of sensitive framework APIs from malicious attacks. It achieves high precision on both datasets without interfering with developers' regular workflows.

C. RQ2: Efficiency and Deployability

To evaluate the practical deployability of DYNAMO, we investigate its impact on the host system's performance and its responsiveness to threats. It is important to note that we do not compare DYNAMO with the baseline methods in this evaluation. The baseline methods are strictly static analysis tools designed to scan model artifacts pre-deployment, whereas DYNAMO is a dynamic, runtime defense mechanism. Comparing the one-time scanning duration of static tools against the continuous runtime overhead of a dynamic monitor is fundamentally inequitable. Therefore, our evaluation focuses on the absolute performance impact of DYNAMO on standard model execution.

We assess efficiency using two primary metrics:

- *Performance Overhead:* The degradation in model loading time and inference throughput (Samples/Second) caused by deploying DYNAMO.
- *Detection Latency:* The time elapsed from the exact moment a malicious operation is initiated by the model to the generation of the final alert (including Autoencoder screening and Large language model (llm) reasoning).

Performance Overhead Analysis. The performance impact of DYNAMO on normal model execution is presented in Table V. The results demonstrate that DYNAMO introduces marginal overhead to the target system. Specifically, the total execution time increases by an average of only 5.69%, with the overhead consistently remaining within a narrow 4.9% to 6.0% margin across fundamentally different model architectures.

TABLE V: Runtime execution overhead and detection latency of DYNAMO evaluated on standard foundation models.

Model	Execution Time (s)			Detection Latency (s)
	Vanilla	w/ DYNAMO	Overhead	
BERT	11.4857	12.1767	6.02 %	13.5014
YamNet	16.3302	17.1434	4.98 %	10.3641
EfficientNet	24.4290	25.8889	5.97 %	6.5911
VGG16	25.0444	26.4766	5.46 %	8.7171
Average	19.3223	20.4214	5.69 %	9.7934

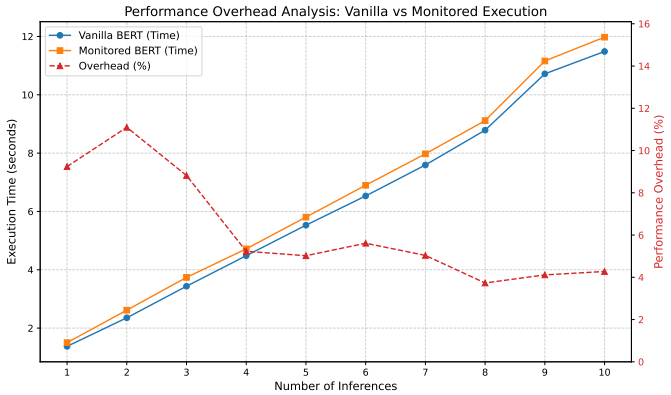

Fig. 3: Performance overhead analysis showing the execution time of Vanilla vs. Monitored BERT and the relative overhead percentage across varying numbers of inferences.

This performance penalty is attributed to our architectural design. First, the application-layer dynamic instrumentation employs a highly lightweight boundary-hooking mechanism. Instead of deeply traversing and serializing complex, multi-gigabyte Tensor objects, it only captures high-level metadata. More importantly, the core analytical engines of DYNAMO—the Autoencoder screening module and the llm reasoning module—run entirely asynchronously on the CPU. Because they do not compete for GPU memory or compute cycles, the intensive matrix multiplications and data parallelism required for AI inference remain completely unhindered.

To further demonstrate that our detection system minimally impacts inference efficiency, we conducted an in-depth analysis of the relationship between the number of inference iterations and the relative latency overhead. Figure 3 illustrates the execution time of the BERT model over 1 to 10 inference steps, comparing the vanilla execution with the DYNAMO-monitored execution. As observed, the relative overhead percentage is highest during the initial executions but quickly drops and stabilizes at a lower baseline as the number of inferences increases.

This phenomenon aligns well with the operating characteristics of deep learning frameworks. The model loading and initialization phases generate a large number of intensive system calls, such as memory mapping operations and heavy file I/O for loading model weights, which incur temporary CPU-

TABLE VI: Ablation study of DYNAMO on detection performance, efficiency, and llm token overhead. Latency represents the average detection time per model, and Token Cost represents the average number of tokens consumed per model.

Method	Precision	Recall	F1	Latency (s)	Token Cost
DYNAMO-S	90.91%	44.30%	59.57%	30.3126	867075
DYNAMO-L	85.25%	**98.99%**	91.61%	**8.9778**	**50005**
DYNAMO	**100.00%**	**98.99%**	**99.49%**	9.7934	57207.1

bound monitoring overhead. However, once the model enters the continuous inference phase, the intensive computations are primarily offloaded to hardware accelerators. These computations generate very few new system calls and do not occupy main CPU resources. Therefore, the overhead introduced by dynamic instrumentation and system call monitoring gradually diminishes.

Detection Latency Analysis. As shown in Table V, DYNAMO achieves an average detection latency of 9.79 seconds. This latency encompasses the full analytical pipeline: capturing the low-level system calls, completing the Autoencoder forward pass, extracting the corresponding Python contextual logs, and generating the final llm intent classification. It is crucial to note that the llm intent classification is only invoked on a small fraction of operations marked as abnormal by the Autoencoder, so its cost does not scale with total runtime but solely with the number of suspicious events.

While dynamic reasoning inherently introduces some delay compared to simple signature matching, this two-stage design keeps the overall overhead well within the acceptable budget for offline model vetting pipelines and pre-deployment security checks. Our experiments suggest that DYNAMO can be seamlessly integrated as the final security gate in the model release pipeline with acceptable latency during pre-deployment security testing.

D. RQ3: Component Ablation and Sensitivity

In this section, we evaluate the contribution of each core component of DYNAMO through ablation studies. To demonstrate the necessity of our two-stage cross-layer architecture, we evaluate two variants of DYNAMO:

- *DYNAMO–S*: which removes the unsupervised Autoencoder screening stage, directly feeding all raw system call windows and application-level instrumentation logs to the llm for anomaly detection;
- *DYNAMO–L*: which removes the application-level dynamic instrumentation, relying solely on the anomalous system call operations filtered by the Autoencoder for the llm to make its final judgment without high-level Python semantic context;

We evaluate the performance of these two variants and compare them with the full DYNAMO model. The evaluation metrics cover detection accuracy, including precision, recall, and F1-score, as well as system efficiency involving detection latency and LLM token cost. Table VI details the comprehensive metrics.

Overall, the results show that both variants of DYNAMO exhibit significantly degraded performance or unacceptable system overhead compared to the full model, proving that both stages are indispensable.

Specifically, DYNAMO-S suffers from a catastrophic decrease in detection efficiency and a massive surge in llm token consumption. Because it bypasses the Autoencoder screening, the llm is overwhelmed by the massive volume of benign, repetitive system events generated during normal model loading and inference. This increases the average detection latency to an impractical level (20.5 seconds) and incurs extreme API costs (809868 tokens).In addition, it slightly reduces precision, as the llm is more prone to hallucination when processing excessive background noise. This demonstrates that the statistical screening stage is crucial for filtering out normal activity and maintaining the practical deployability of the system.

On the other hand, DYNAMO-L performs the worst in terms of detection accuracy, particularly suffering a severe drop in precision (14.75% lower than the full model). By removing the application-level instrumentation logs, the llm is forced to judge intent based entirely on low-level system calls. Without the high-level semantic context such as function arguments to explain why a system call was executed, benign operations are frequently misclassified as malicious network exfiltration. This confirms our prior observation that a severe semantic gap exists at the OS level, and dynamic cross-layer context alignment is the most important component for eliminating false positives and accurately discerning the true intent of AI models.

VI. Related Work

Malicious Code Attacks in the AI Supply Chain. Attackers embed malicious code into model artifacts on open-source hubs. Early work targets insecure serialization: Python's `pickle` can execute arbitrary code during deserialization [26]. Static tools such as Fickling [7], Picklescanning [8], and ModelScan [6] integrated with ClamAV [33] detect malware signatures by decompiling bytecodes or matching patterns, but are susceptible to evasion and produce high false positives when legitimate operations resemble malicious patterns. Recent attacks have evolved to framework-native methodologies that abuse built-in APIs for stealthy execution. Zhu et al. [11] introduced TensorAbuse, embedding persistent attacks in the model's computational graph via TensorFlow APIs. Using officially supported operations, these attacks bypass static scanners. While the authors proposed an LLM-based tool to statically classify API capabilities, it struggles with semantic ambiguity in distinguishing benign from malicious intents. In contrast, DYNAMO employs dynamic, cross-layer runtime monitoring with high-level execution context to accurately identify malicious intent.

Defenses Against AI Supply Chain Attacks. Current defenses for AI model supply chain attacks primarily rely on static analysis and signature-based scanning. These tools inspect model artifacts for known malicious patterns without

analyzing dynamic execution, failing to establish semantic context. A benign component fetching remote weights and a malicious payload establishing a reverse shell might invoke the same library, inevitably leading to high false positive and false negative rates. Furthermore, these defenses are inherently reactive, engineered for specific, previously observed vulnerabilities. This severely limits their ability to counter zero-day exploits or novel attack methodologies that do not conform to known signatures.

VII. Discussion

Scope of detection. DYNAMO is designed to detect malicious behaviors that a model artifact executes against the host system, such as data exfiltration. Its detection pipeline relies on observing the actual runtime interactions between the model process and the operating system. DYNAMO could not address attacks that target the model itself without affecting the host system. In particular, model-level threats such as neural backdoor injection fall outside our detection scope. These attacks embed malicious functionality into the model's learned parameters, and at inference time the compromised model performs only normal tensor computations through standard framework operations. So these attacks generate no anomalous system calls or suspicious API invocations that DYNAMO relies upon as its primary detection signal.

Consequently, these attacks are invisible to DYNAMO's cross-layer monitoring pipeline. Such attacks compromise the model's prediction integrity rather than its runtime behavior, and their malicious effects manifest as incorrect or biased outputs rather than as anomalous system calls or API invocations. Defending against such threats requires complementary techniques such as model provenance verification, weight integrity checking, and robust training, which are orthogonal to and compatible with DYNAMO's host-level defense.

VIII. Conclusion

Malicious model artifacts present a growing threat to the AI supply chain: attackers can embed harmful payloads within serialized models that execute during loading or inference, evading traditional static scanners by using legitimate framework APIs. In this paper, we propose DYNAMO, a hybrid runtime defense system that combines non-bypassable kernel-level system-call monitoring with LLM-assisted Python semantic instrumentation to detect malicious behaviors in model artifacts. Instead of relying solely on static signature matching, DYNAMO captures the actual runtime execution evidence through a two-stage architecture: an unsupervised Autoencoder prefilter that surfaces anomalous low-level operations, and a cross-layer LLM reasoning engine that aligns these operations with their high-level semantic context to determine malicious intent. We evaluate DYNAMO on the MalHug Dataset and advanced synthetic attacks. The results show that DYNAMO achieves near-perfect detection, while introducing an average runtime overhead of only 5.69%, demonstrating that DYNAMO provides a practical, deployable defense for the AI model supply chain.

References

[1] H. Face, "Hugging face – the ai community building the future." https://huggingface.co/, 2026, https://huggingface.co/.

[2] P. Girnus, F. Tucci, D. Patel, S. Dulude, A. Verma, and J. R. Navato, "Your ai gateway was a backdoor: Inside the litellm supply chain compromise," https://www.trendmicro.com/en_us/research/26/c/inside-litellm-supply-chain-compromise.html, 2026, access:2026-05-18. [Online]. Available: https://www.trendmicro.com/en_us/research/26/c/inside-litellm-supply-chain-compromise.html

[3] P. P. Index, "Incident report: Litellm/telnyx supply-chain attacks, with guidance," https://blog.pypi.org/posts/2026-04-02-incident-report-litellm-telnyx-supply-chain-attack/, 2026, access:2026-05-18. [Online]. Available: https://blog.pypi.org/posts/2026-04-02-incident-report-litellm-telnyx-supply-chain-attack/

[4] unit42, "Weaponizing the protectors: Teampcp's multi-stage supply chain attack on security infrastructure," https://origin-unit42.paloaltonetworks.com/teampcp-supply-chain-attacks/, 2026, access:2026-05-18. [Online]. Available: https://origin-unit42.paloaltonetworks.com/teampcp-supply-chain-attacks/

[5] J. Zhao, S. Wang, Y. Zhao, X. Hou, K. Wang, P. Gao, Y. Zhang, C. Wei, and H. Wang, "Models are codes: Towards measuring malicious code poisoning attacks on pre-trained model hubs," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 2087–2098. [Online]. Available: https://doi.org/10.1145/3691620.3695271

[6] Protect AI, "Modelscan: Protection against model serialization attacks," 2021, access:2026-05-18. [Online]. Available: https://github.com/protectai/modelscan

[7] Trail of Bits, "fickling," 2021, access:2026-05-18. [Online]. Available: https://github.com/trailofbits/fickling

[8] Hugging Face, "Pickle scanning," 2024, access:2026-05-18. [Online]. Available: https://huggingface.co/docs/hub/security-pickle#pickle-scanning

[9] A. Virkud, M. A. Inam, A. Riddle, J. Liu, G. Wang, and A. Bates, "How does endpoint detection use the MITRE ATT&CK framework?" in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 3891–3908. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/virkud

[10] Z. Jia, Y. Xiong, Y. Nan, Y. Zhang, J. Zhao, and M. Wen, "MAGIC: Detecting advanced persistent threats via masked graph representation learning," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 5197–5214. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/jia-zian

[11] R. Zhu, G. Chen, W. Shen, X. Xie, and R. Chang, " My Model is Malware to You: Transforming AI Models into Malware by Abusing TensorFlow APIs ," in *2025 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 449–466. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP61157.2025.00012

[12] L. Gambacorta and V. Shreeti, "The ai supply chain," *Review of Network Economics*, vol. 24, no. 4, pp. 205–223, 2025. [Online]. Available: https://doi.org/10.1515/rne-2025-0063

[13] Hugging Face, "Models - hugging face," https://huggingface.co/models, 2026.

[14] W. Jiang, N. Synovic, M. Hyatt, T. R. Schorlemmer, R. Sethi, Y.-H. Lu, G. K. Thiruvathukal, and J. C. Davis, "An empirical study of pre-trained model reuse in the hugging face deep learning model registry," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 2463–2475.

[15] M. Taraghi, G. Dorcelus, A. Foundjem, F. Tambon, and F. Khomh, "Deep learning model reuse in the huggingface community: Challenges, benefit and trends," in *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2024, pp. 512–523.

[16] M. L. Siddiq, T. H. Romel, N. Sekerak, B. Casey, and J. C. S. Santos, "An empirical study on remote code execution in machine learning model hosting ecosystems," 2026. [Online]. Available: https://arxiv.org/abs/2601.14163

[17] HiddenLayer, "Ai threat landscape 2026," https://www.hiddenlayer.com/report-and-guide/threatreport2026, 2026, access:2026-05-18. [Online]. Available: https://www.hiddenlayer.com/report-and-guide/threatreport2026

[18] Z. Wang, C. Liu, X. Cui, J. Yin, and X. Wang, "Evilmodel 2.0: Bringing neural network models into malware attacks," *Computers & Security*, vol. 120, p. 102807, 2022. [Online]. Available: http://dx.doi.org/10.1016/j.cose.2022.102807

[19] A. Kathikar, A. Nair, B. Lazarine, A. Sachdeva, and S. Samtani, "Assessing the vulnerabilities of the open-source artificial intelligence (ai) landscape: A large-scale analysis of the hugging face platform," in *2023 IEEE International Conference on Intelligence and Security Informatics (ISI)*, 2023, pp. 1–6.

[20] W. Jiang, N. Synovic, R. Sethi, A. Indarapu, M. Hyatt, T. R. Schorlemmer, G. K. Thiruvathukal, and J. C. Davis, "An empirical study of artifacts and security risks in the pre-trained model supply chain," in *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, ser. SCORED'22. New York, NY, USA: Association for Computing Machinery, 2022, p. 105–114. [Online]. Available: https://doi.org/10.1145/3560835.3564547

[21] N.-J. Huang, C.-J. Huang, and S.-K. Huang, "Pain pickle: Bypassing python restricted unpickler for automatic exploit generation," in *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*, 2022, pp. 1079–1090.

[22] T. A. Nguyen, T. H. M. Le, and M. A. Babar, "Securing the ai supply chain: What can we learn from developer-reported security issues and solutions of ai projects?" 2026. [Online]. Available: https://arxiv.org/abs/2512.23385

[23] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf

[24] N. Koutroumpouchos, G. Lavdanis, E. Veroni, C. Ntantogian, and C. Xenakis, "Objectmap: detecting insecure object deserialization," in *Proceedings of the 23rd Pan-Hellenic Conference on Informatics*, ser. PCI '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 67–72. [Online]. Available: https://doi.org/10.1145/3368640.3368680

[25] A. D. Kellas, N. Christou, W. Jiang, P. Li, L. Simon, Y. David, V. P. Kemerlis, J. C. Davis, and J. Yang, "Pickleball: Secure deserialization of pickle-based machine learning models," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 3341–3355. [Online]. Available: https://doi.org/10.1145/3719027.3765037

[26] B. Casey, J. C. S. Santos, and M. Mirakhorli, "A large-scale exploit instrumentation study of ai/ml supply chain attacks in hugging face models," 2024. [Online]. Available: https://arxiv.org/abs/2410.04490

[27] Python Software Foundation, "pickle — python object serialization," https://docs.python.org/3/library/pickle.html, 2026, accessed: 2026-05-25.

[28] T. Liu, G. Meng, P. Zhou, Z. Deng, S. Yao, and K. Chen, "The art of hide and seek: Making pickle-based model supply chain poisoning stealthy again," 2025. [Online]. Available: https://arxiv.org/abs/2508.19774

[29] The HDF Group, "High-performance data management and storage suite," https://www.hdfgroup.org/HDF5/, accessed: 2026-05-25.

[30] safetensors, "Safetensors," https://github.com/huggingface/safetensors, accessed: 2026-05-25.

[31] onnx, "ONNX," https://github.com/onnx/onnx, 2020, accessed: 2026-05-25.

[32] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: A system for Large-Scale machine learning," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, pp. 265–283. [Online]. Available: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi

[33] Cisco Systems, "ClamAVNet," https://www.clamav.net/, access:2026-05-25. [Online]. Available: https://www.clamav.net/

[34] H. Siadati, S. Jafarikhah, E. Sahin, T. Hernandez, E. Tripp, D. Khryashchev, and A. Kharraz, "Devphish: Exploring social engineering in software supply chain attacks on developers," in *2024 IEEE 15th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, 2024, pp. 517–523.

[35] S. Yun, Y. Zhang, Z. Shi, L. Wang, Y. Wang, and Y. Bai, "Large-scale empirical study of secret keys leakage in hugging face spaces," 05 2026.

[36] M. Dimjašević, S. Atzeni, I. Ugrina, and Z. Rakamaric, "Evaluation of android malware detection based on system calls," in *Proceedings of the 2016 ACM on International Workshop on Security And Privacy Analytics*, ser. IWSPA '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 1–8. [Online]. Available: https://doi.org/10.1145/2875475.2875487

[37] S. Shakya and M. Dave, "Analysis, detection, and classification of android malware using system calls," 2022. [Online]. Available: https://arxiv.org/abs/2208.06130

[38] David Cohen, "Data scientists targeted by malicious hugging face ml models with silent backdoor," https://jfrog.com/blog/data-scientists-targeted-by-malicious-hugging-face-ml-models-with-silent-backdoor/, 2024.

IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove the template text from your paper may result in your paper not being published.